DATA 266 - Homework 7 Findings

GANs, and Autoencoders vs. Variational Autoencoders on FashionMNIST

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0

These are calculated with the same Step 0 formulas as Assignment 1. SEED 9486 is used for Python, NumPy, and PyTorch, the train/validation split, DataLoader shuffling, the VAE's sampling noise, and the latent vectors used for generation. SLICE picks the images shown in the reconstruction figures (the first test image of each class at index 486 or later). The assignment says the second hyperparameter configuration can be skipped because two different models are already trained, so HP_ID, CLS_A, and CLS_B are only reported.

Introduction

This homework has a written part and a coding part. Questions 1 to 3 cover how GANs work, why they suffer from mode collapse, and three important GAN variants. For Question 4, I trained a fully connected autoencoder (FC-AE) and a variational autoencoder (VAE) on FashionMNIST in PyTorch and compared them on reconstruction and generation. I used the professor's Demo_7_GAN.ipynb as the reference for the GAN answers, and Demo_7_AEandVAE.ipynb as the starting point for the code. I kept its architectures, preprocessing, and training loops, and moved them from MNIST to FashionMNIST. Every number in this report comes from the executed HW7_GAN_AE_VAE.ipynb notebook, run top to bottom on CPU (Apple M1). I ran it twice and got exactly the same losses and test metrics both times.

What's in this submission

Question	What was completed
Step 0	Personal parameters calculated from SID4 and checked, everything seeded, deterministic algorithms turned on.
Question 1	GAN architecture: the roles of the generator and discriminator, how they are trained in turns, the minimax objective, and its theoretical goal ($p_g = p_{data}$).
Question 2	Mode collapse: what it is, why it happens, and two mitigations (minibatch discrimination / minibatch standard deviation, and WGAN-GP).
Question 3	Three GAN variants: DCGAN, StyleGAN, and Conditional GAN, each with the problem it solves and what it changes.

Question 4	FC-AE and VAE trained on FashionMNIST with the same size and settings, loss curves, separate VAE reconstruction/KL/total curves, reconstructions, 64 samples from the VAE prior, test-set evaluation, and a comparison.

Question 1: GAN Architecture

A GAN is made of two networks trained against each other. The **generator G** takes a random noise vector z (for example 100 values from N(0, I)) and turns it into a fake image G(z). In Demo_7_GAN it is an MLP, 100 to 256 to 512 to 1024 to 784, with tanh at the output. G never sees a real image; it only learns from the discriminator's feedback. The **discriminator D** is a binary classifier that outputs how likely an image is to be real. In the demo it is 784 to 512 to 256 to 1 with LeakyReLU(0.2), trained with BCEWithLogitsLoss.

Training alternates on every batch. First D is updated: real images get label 1 and fakes get label 0, and the fakes are detached so G does not change. Then G is updated: new fakes are passed through D and the loss is computed as if they were real, so the gradient flows back through D into G and pushes G toward images that D calls real. It works like a counterfeiter and a detective: each one improving forces the other to improve. Goodfellow et al. (2014) write this as a minimax game:

$$\min_G \max_D V(D, G) = E_{x \sim p_data}[\log D(x)] + E_{z \sim p_z}[\log(1 - D(G(z)))]$$

D wants V to be large (real images near 1, fakes near 0) and G wants it to be small (D(G(z)) near 1). In practice G maximizes log D(G(z)) instead, because early in training D(G(z)) is close to 0 and log(1 - D(G(z))) gives almost no gradient. The demo does this with criterion(D(fake), real_labels).

Theoretical goal. For a fixed G, the best discriminator is $D^*(x) = p_{data}(x) / (p_{data}(x) + p_g(x))$. Plugging it in gives $V = -\log 4 + 2\,\mathrm{JSD}(p_{data} \,||\, p_g)$, which is smallest only when $p_g = p_{data}$. So the goal is for G to learn the real data distribution exactly. At that point D(x) = 1/2 everywhere and D can no longer tell real from fake. In practice this equilibrium is hard to reach, which is where problems like mode collapse come from.

Question 2: Mode Collapse

What it is. Mode collapse is when the generator only produces a few kinds of outputs instead of the full variety in the data. On FashionMNIST, a collapsed GAN might only make sneakers, or the same few T-shirts, whatever z it gets.

Why it happens. G is only rewarded for fooling the current D, not for being diverse. If one output fools D well, G pushes many z values toward it. D then learns to reject it, G jumps to another output, and the two keep chasing each other instead of converging. D also judges each image on its own, so it cannot easily see that a whole batch of fakes looks the same.

Mitigation 1: minibatch discrimination / minibatch standard deviation. D is given features computed across the batch, such as how similar the samples are to each other (Salimans et al., 2016) or the average standard deviation of the features (ProGAN and StyleGAN). A collapsed batch has very little variation, so D spots it easily, and G gets a direct gradient toward more diverse outputs.

Mitigation 2: WGAN-GP. D is replaced by a critic that estimates the Wasserstein distance between the real and fake distributions (Arjovsky et al., 2017), and a gradient penalty keeps the critic

1-Lipschitz (Gulrajani et al., 2017). With the original loss (JS divergence), G gets almost no useful gradient when the two distributions barely overlap. The Wasserstein distance still changes smoothly there, so training is more stable and G keeps getting pushed to cover the whole distribution instead of one mode. Other common tricks are spectral normalization, one-sided label smoothing, and unrolled GANs.

Question 3: Three GAN Variants

Variant	Problem it addresses	What it changes	Why it helps
DCGAN (Radford et al., 2015)	Fully connected GANs like the demo's ignore the 2D structure of images and were unstable; plain CNNs did not train reliably either.	Strided convolutions in D and transposed convolutions in G instead of pooling, BatchNorm in both, no fully connected hidden layers, ReLU in G (tanh output) and LeakyReLU in D, Adam with lr 2e-4 and beta1 = 0.5.	Convolutions learn local patterns with far fewer parameters, and BatchNorm and LeakyReLU keep gradients stable. The demo already uses DCGAN's Adam settings and activations, just with linear layers.
StyleGAN (Karras et al., 2019)	z goes straight into the first layer, so pose, color, and texture are mixed together and hard to control separately.	A mapping network (8-layer MLP) turns z into w; the generator starts from a learned constant and w is injected at every layer through AdaIN; per-pixel noise; style mixing during training.	w is more disentangled than z, and per-layer injection separates coarse features (pose, shape) from fine ones (color, texture), giving high quality and much better control.
Conditional GAN (Mirza and Osindero, 2014)	A vanilla GAN cannot choose what it generates; with the demo's GAN you cannot ask for a sneaker.	G and D both get extra information y, usually a class label (one-hot or embedding) concatenated to the input, so the game uses G(z | y) and D(x | y).	G learns p(x | y), so the class can be chosen when generating, and D checks that the image is real and matches its label. The labels also give extra signal that tends to stabilize training.

Question 4: FC-AE and VAE on FashionMNIST

Setup

Item	Value
Data	FashionMNIST from torchvision: 54,000 train / 6,000 validation (seeded 90/10 split of the 60,000 training images) / 10,000 test. ToImage, ToDtype(float32, scale=True) to get pixels in [0, 1], then flatten to 784.
Tensor shapes	image [784], batch [256, 784], inside both models [B, 784] to [B, 256] to [B, 32] to [B, 256] to [B, 784]; the VAE encoder outputs mu and logvar, both [B, 32]
FC-AE	Linear(784, 256) + ReLU, Linear(256, 32) | Linear(32, 256) + ReLU, Linear(256, 784) + Sigmoid; 419,120 parameters; loss = BCE
VAE	same layers, but two encoder heads for mu and logvar, z = mu + exp(0.5 logvar) * eps with eps ~ N(0, I); 427,344 parameters; loss = BCE + KL

Training (both)	Adam lr 1e-3, batch 256 (211 updates per epoch), 20 epochs, about 2 minutes each on CPU (114.3 s and 120.7 s)
Loss units	BCE summed over the 784 pixels and averaged per image (plus KL summed over the 32 latent dimensions for the VAE), so both models are on the same scale

Compared with the demo, I fixed a few things so the comparison is fair: the demo's train_autoencoder ignored its batch_size argument, it added the loss tensor instead of loss.item() (keeping the graph around), the VAE was built with latent size 32 but labeled latent_dim=2, and the AE and VAE losses were on different scales (per-pixel mean vs. per-batch sum). Checks that passed in the reported run: all tensor shapes, pixels in [0, 1], the KL of N(0, I) against the prior is 0 and with mu = 1, logvar = 0 it is exactly 16 (0.5 per dimension), and the VAE total loss equals reconstruction plus KL.

How the two models work

FC-AE. The encoder squeezes 784 pixels into 32 numbers and the decoder rebuilds the image from only those. Because the pixels are in [0, 1] and the output is a sigmoid, each output pixel can be treated as a probability, so binary cross entropy is a natural reconstruction loss. FashionMNIST pixels are not exactly 0 or 1, so the BCE can never reach 0; only the difference between models matters.

VAE. The encoder outputs a distribution N(mu, sigma^2) for each image instead of a single point. You cannot backpropagate through random sampling, so the reparameterization trick computes z = mu + sigma * eps with eps ~ N(0, I), which makes z differentiable with respect to mu and logvar. The loss is the negative ELBO: the same BCE reconstruction term as the FC-AE plus KL(q(z|x) || N(0, I)) = -0.5 * sum(1 + logvar - mu^2 - exp(logvar)).

Role of the KL term. Without it, the VAE would act like a normal autoencoder: sigma would shrink to 0 and the codes would spread out with gaps between them. The KL term pulls mu toward 0 and sigma toward 1, so the codes fill the region where N(0, I) puts its probability. That is what makes **generation** possible: a random z ~ N(0, I) lands somewhere the decoder has learned to decode. **Reconstruction** starts from a real image and rebuilds that same image, which both models can do. Generation has no input image and only runs the decoder, which only the VAE is trained for.

Main results

Metric (per image unless noted)	FC-AE	VAE
Test reconstruction BCE, deterministic (VAE decodes z = mu)	**214.20**	223.76
Test MSE per pixel, deterministic	**0.009698**	0.013927
Test VAE objective with sampled z: recon / KL / total	-	228.35 / 16.59 / 244.95
Final train loss (epoch 20)	212.38	243.04 (recon 226.94 + KL 16.10)
Final validation loss (epoch 20)	212.24	243.07 (recon 226.49 + KL 16.58)

For the VAE I report two kinds of test numbers. The sampled-z numbers are its own training objective. The z = mu numbers remove the sampling noise and are computed exactly like the FC-AE's, so those are the fair reconstruction comparison.

Training behavior

Model	Epoch 1 (train / val)	Epoch 20 (train / val)
FC-AE: BCE	288.35 / 242.66	212.38 / 212.24
VAE: total	317.39 / 276.29	243.04 / 243.07
VAE: train recon / train KL	303.64 / 13.74	226.94 / 16.10

Both models trained smoothly with no sign of overfitting. The FC-AE's validation loss was below its training loss at every epoch (partly because the training loss is averaged while the model is still improving during the epoch), and it was still dropping by about 0.30 per epoch at the end. For the VAE, the reconstruction term kept falling (238.36 at epoch 5 to 226.94 at epoch 20) while the training KL slowly rose (15.54 to 16.10), so as the decoder got better, the encoder started putting a bit more information into z, which the KL term charges for. The validation KL moved around between 15.13 and 16.58 from epoch to epoch while the reconstruction term moved the other way, so the validation total stayed smooth. It was lowest at epoch 19 (242.86) and essentially flat at epoch 20.

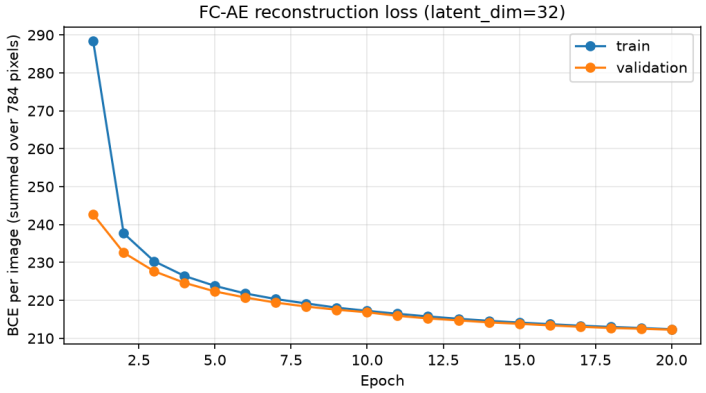


Figure 1. FC-AE training and validation BCE per image.

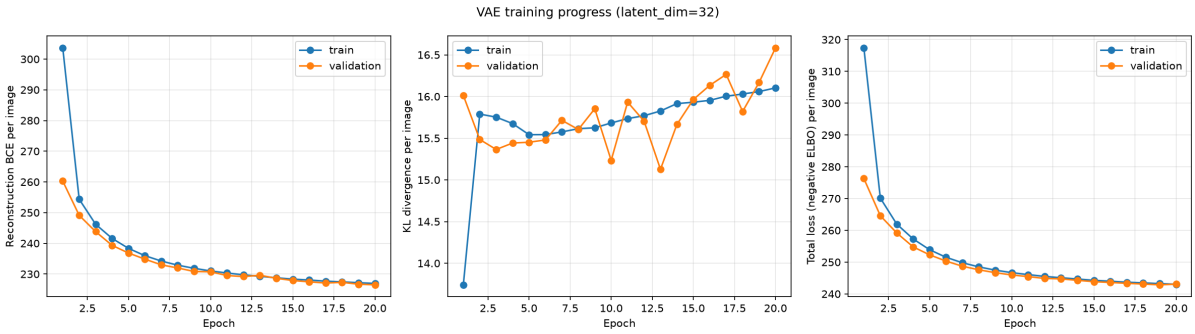


Figure 2. VAE reconstruction BCE, KL divergence, and total loss per image, for training and validation.

Reconstructions and generated samples

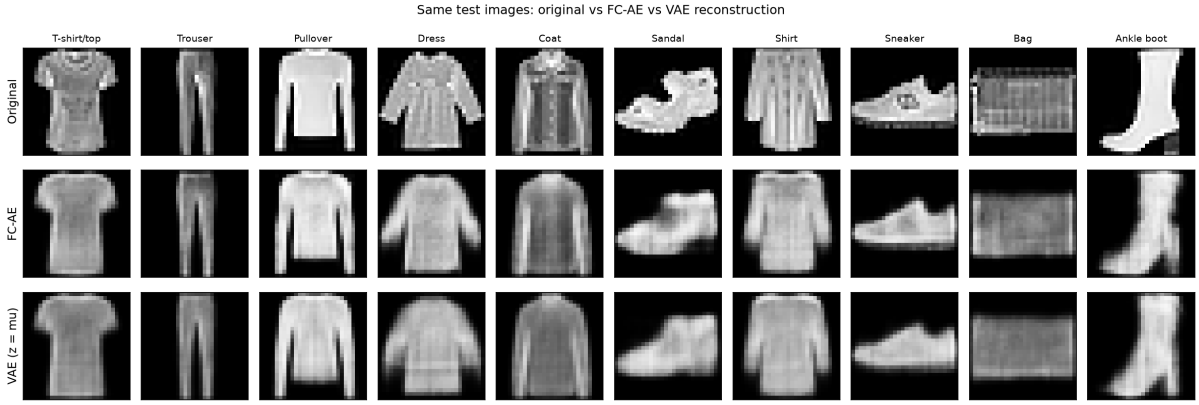


Figure 3. The same 10 test images (one per class, chosen with SLICE): originals, FC-AE reconstructions, and VAE reconstructions (z = mu).

Both models get the overall shape and brightness of every item right (the trouser legs, the pullover's sleeves, the boot's heel), and both lose fine detail: neither keeps the T-shirt print, the coat's buttons, the sandal's straps (both turn it into a solid shoe-like shape), or the bag's pattern. The difference is mostly sharpness. The FC-AE's outputs have slightly crisper edges and a bit more inner texture, while the VAE's are smoother and look more like an average item of that class. Both are blurrier than the originals because a 32-number code plus a pixel-wise loss favors predicting the average brightness where the model is unsure.



Figure 4. 64 images decoded by the VAE from random z ~ N(0, I) (seeded).

The prior samples are mostly recognizable FashionMNIST items: T-shirts, pullovers and coats, trousers, ankle boots, and sneakers, with upper-body garments showing up most often. They are blurry, a few are ambiguous blobs, some look like a mix of two classes, and a few have horizontal

stripe artifacts. For a small fully connected VAE trained for 20 epochs that is a reasonable result, and it shows the KL term did its job. I did not sample from the FC-AE, because its latent space has no prior and it was never trained to generate.

Comparison

The FC-AE is better at reconstruction. Measured the same way, its test BCE is about 9.6 lower per image (214.20 vs 223.76) and its per-pixel MSE is about 30% lower (0.0097 vs 0.0139). That is expected, since reconstruction is its only objective, while the VAE also pays about 16.6 per image for the KL term, and its reconstruction gets worse again (228.35) when z is sampled instead of set to mu. In return, the VAE is a real generative model: it can produce new, mostly plausible clothing images from random noise, which the FC-AE cannot. So the FC-AE is the better compressor and the VAE is the better generator.

What Could Improve Each Model

FC-AE: convolutional encoder and decoder layers to keep the 2D structure and fine textures, and more epochs, since the loss was still dropping at epoch 20.

VAE: the same convolutional layers and longer training, plus tuning the KL weight (a beta-VAE) or the latent size to balance sharper reconstructions against better samples. None of this was tested here, since the assignment asked for one configuration per model.

Limitations

This is one seed. Two full runs gave identical numbers, but a different seed or a GPU would change them slightly. Both models are small MLPs (about 0.42M parameters) trained for only 20 epochs with no tuning, so the comparison is "same settings for both", not "each model at its best". BCE and MSE only measure pixel similarity and reward blurry average predictions. They say nothing about how realistic the generated samples are (that would need something like FID), so my judgment of the samples is visual. Questions 1 to 3 are written answers based on the papers and the demo; I did not train a GAN in this homework.

Final findings

GANs train a generator and a discriminator against each other, with the goal that $p_g = p_{data}$. Mode collapse is a common failure, which minibatch statistics and WGAN-GP help with, and DCGAN, StyleGAN, and Conditional GAN each fix a specific weakness of the vanilla GAN. On FashionMNIST, the FC-AE reconstructed better (test BCE 214.20 vs 223.76, MSE 0.0097 vs 0.0139) and slightly more sharply, while the VAE gave up some reconstruction quality for its KL term (about 16.6 per image) and in return could generate new, mostly recognizable clothing from random noise. The main takeaway is the trade-off between exact reconstruction and a smooth latent space you can sample from.